# **AI**-TECH

Przykład 3 DO LABORATORIUM 1

## z UCZENIA MASZYNOWEGO

## Klasyfikacja nadzorowana

**Szymon Zaporowski**

**Regresja logistyczna, SVM i Drzewa Losowe - wykorzystanie danych z MNIST i tensorflow do porównania klasyfikatorów**

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)"

---

# 1. Przykład 3

Aby porównać działanie Regresji Logistycznej i SVM oraz algorytmu lasów losowych posłużymy się przykładem, który został wykorzystany w trakcie poprzedniego laboratorium i pierwotnie dotyczył tylko regresji liniowej. Ponownie wykorzystany zostanie zbiór danych MNIST, który zawiera 70 tysięcy obrazków o rozdzielczości 28x28 w odcieniach szarości, który został zebrany przez Yann'a Lecun (http://yann.lecun.com/exdb/mnist/). Obrazki te zawierają zeskanowane zapisy cyfr: 0-9. Daje to do dyspozycji 10 klas (0-9). Zbiór ten można wczytać na wiele róznych sposobów do projektu, ale żeby uniknąć w tym momencie pewnego preprocessingu skorzystamy z możliwości biblioteki tensorflow.

Podobnie jak poprzednio, do celów aktualnego ćwiczenia przemodelujemy opis klas do problemu binarnego w którym cyfry od 0 do 8 będą stanowić klasę 0, zaś klasę 1 będzie stanowić cyfry 9.

In [17]:
```
# Import podstawowych bibliotek
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import keras
import tensorflow as tf
from sklearn.linear_model import LogisticRegression
```

```
from io import StringIO
import requests
from random import randint
from sklearn import svm #Zaimporotowanie modelu SVM z packietu sklearn
from sklearn import metrics #Import modułu metrics z pakietu scikit-lear do policzenia miary dokładności klasyfikatora
import seaborn as sns
import pandas as pd
print(tf.__version__)
print('keras:',keras.__version__)
```

```
2.19.0
keras: 3.9.2
```

W takim rozwiązaniu, zbiór wejściowy jest wczytywany od razu z podziałem na zbiór treningowy (60000) oraz testowy (10000). Zbiór testowy zostanie wykorzystany później do oceny treningu i wyznaczenia miar błędów.

In [18]:
```
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data(path='mnist.npz')

print("Training set size: ", X_train.shape)
print("Test set size: ", X_test.shape)
```

```
Training set size:  (60000, 28, 28)
Test set size:  (10000, 28, 28)
```

Do celów aktualnego ćwiczenia przemodelujemy opis klas, tak aby uzyskać problem binarny, w którym cyfry od 0 do 8 będą stanowić klasę 0, zaś klasę 1 będzie stanowić cyfry 9.

In [19]:
```
y_train_binary = np.where(y_train < 9, 0.0, 1.0).reshape(y_train.shape[0], 1)
y_train_binary = y_train_binary.astype('float32')
```

Dane wejściowe traktujemy jako zbiór cech, stąd "spłaszczamy" reprezentację 2D do reprezentacji 1D (28*28 -> 784). Taka spłaszczona reprezentacja obrazka przemnożona przez ilość obrazków występujących w rozważanym zbiorze daje nam wektor, który jest łatwiejszy do przetwarzania przez algorytmy uczenia maszynowego. Wartości te zostają również znormalizowane do przedziału [0,1].

In [20]:
```
#przygotowanie danych treningowych jako wektor znormalizowany
X_train_flat = X_train.reshape((X_train.shape[0], X_train.shape[1]*X_train.shape[2]))
X_train_flat = X_train_flat.astype('float32')/255.0
print("Input dataset shape: ", X_train_flat.shape)

#przygotowanie danych testowych jako wektor znormalizowany
y_test_binary = np.where(y_test < 9, 0.0, 1.0).reshape(y_test.shape[0], 1)
y_test_binary = y_test_binary.astype('float32').flatten()
X_test_flat = X_test.reshape((X_test.shape[0], X_test.shape[1]*X_test.shape[2]))
```

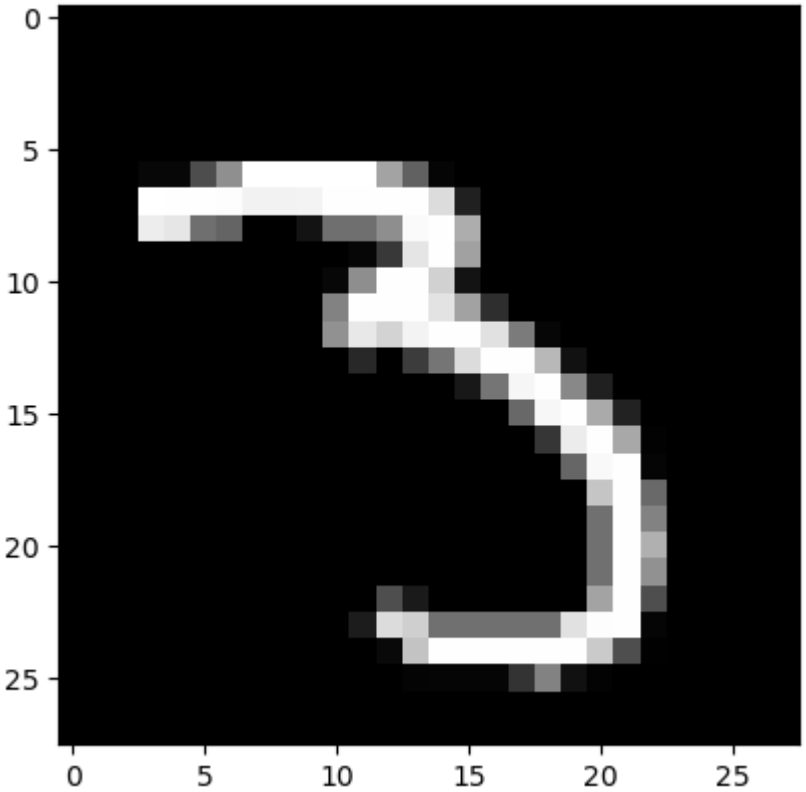
```
X_test_flat = X_test_flat.astype('float32')/255.0
print("Test dataset shape: ", X_test_flat.shape)
```

```
Input dataset shape:  (60000, 784)
Test dataset shape:  (10000, 784)
```

Przykładowy obraz z bazy danych, jego oryginalna etykieta oraz aktualna przynależność do klasy została przedsatwiona w poniższej sekcji.

In [21]:
```
example = randint(0, 100)
plt.imshow(X_train[example], cmap="gray")
print("True orginal label :", y_train[example])
print("True binary label :", y_train_binary[example])
```

```
True orginal label : 3
True binary label : [0.]
```




W tym kroku dopasowujemy model regresji logistycznej do "spłaszczonych" danych wejściowych oraz prezentujemy wynik predykcji dla zbioru danych testowych

In [22]:
```
model = LogisticRegression(solver='newton-cg',random_state=0).fit(X_train_flat, y_train_binary.flatten())
```

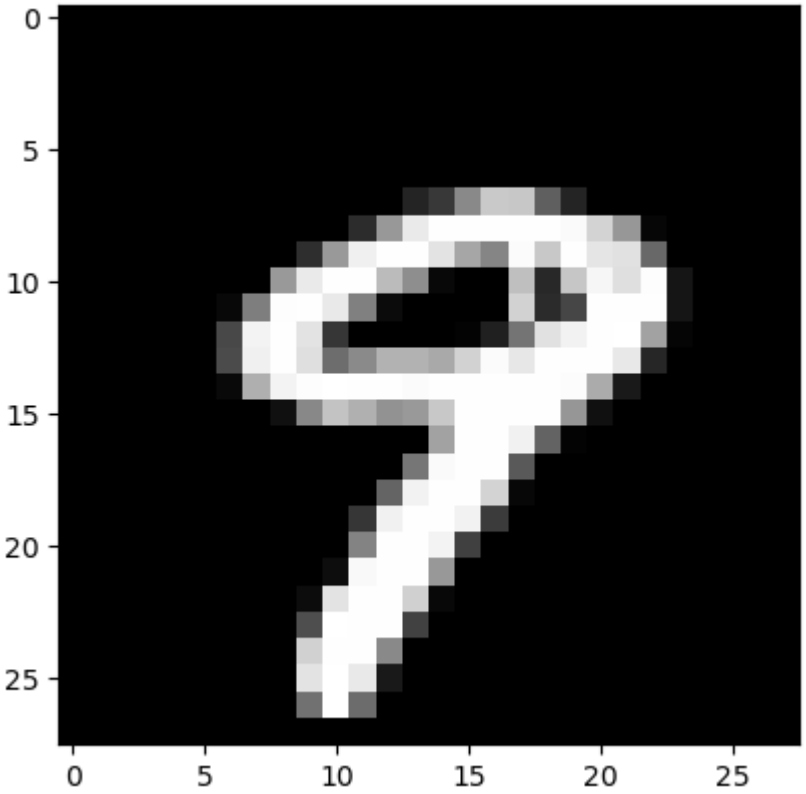
```
print("Wynik ewaluacji wytrenowanego modelu: ", model.score(X_test_flat, y_test_binary.flatten()))
```

```
Wynik ewaluacji wytrenowanego modelu:  0.9677
```

Mając wytrenowany model można go użyć do oceny prognozy $p$ innego wektora wejściowego. Dodatkowo doknamy predykcji dla całego zbioru testowego.

In [23]:
```
test_example = 9
p = model.predict(X_test_flat[test_example:test_example+1])

# Wyswietlenie wartości prognozy ("prawdopodobieństwa") p z przedziału [0,1]
print("Wartość prognozy dla wybranego przypadku: ", p)
print("Prawdziwa etykieta klasy dla wybranego przypadku: ", int(y_test_binary[test_example]))
#Predykcja odpowiedzi modelu na dane testowe

# Użycie wartości progu w celu przypisania do odpowiednich klas
predicted_class = 0 if p < 0.5 else 1
print("Przewidywana etykieta klasy dla wybranego przypadku: ", predicted_class)

plt.imshow(X_test[test_example], cmap="gray")

y_pred = model.predict(X_test_flat) # Predykcja dla całego zbioru testowego
```

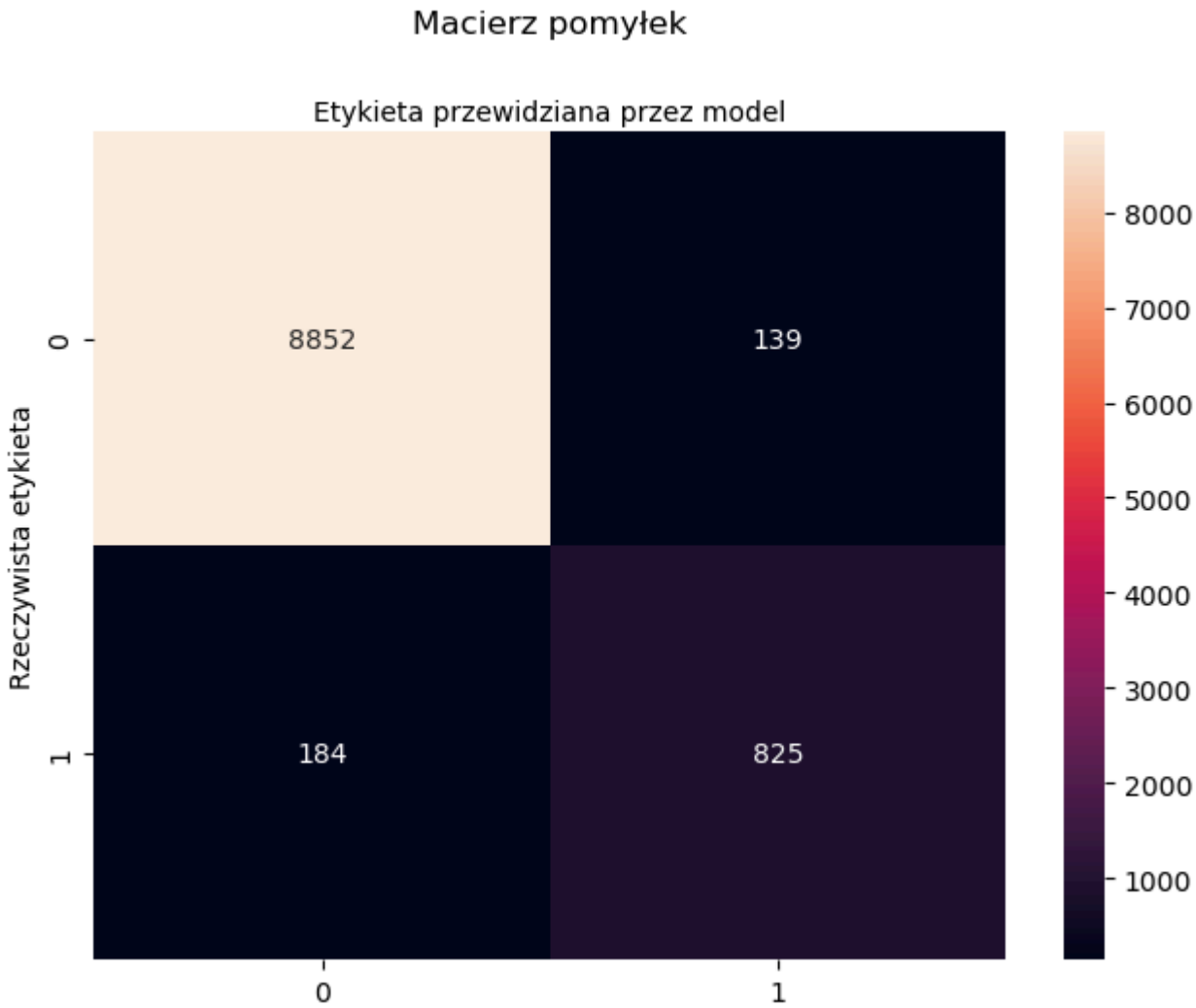
```
Wartość prognozy dla wybranego przypadku:  [1.]
Prawdziwa etykieta klasy dla wybranego przypadku:  1
Przewidywana etykieta klasy dla wybranego przypadku:  1
```




Sprawdzimy jak wyglądają wyniki z wykorzystaniem macierzy pomyłek

In [24]:
```
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = metrics.confusion_matrix(y_test_binary, y_pred)
cnf_matrix
class_names=[0,1] # nazwy klas
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[24]:
```
Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')
```




Po macierzy pomyłek widać, że zgodnie z założeniami otrzymujemy wyniki tylko w obrębie dwóch klas - 0 oraz 1. Można jednak zauważyć, że przez dokonanie ingerencji w klasy obecnie klasa 0 zawiera znacznie więcej przykładów niż klasa 1. Do czego to prowadzi będzie dokładnie widać po obliczeniu metryk precission i recall. Inicjalizujemy klasyfiokator oparty o SVM, podobnie jak to było w przykładzie 1.

In [25]:
```
clf = svm.SVC(C=10, kernel='poly',degree=5, probability=False, max_iter=10, decision_function_shape='ovo')
#Trening modelu z wykorzystaniem danych treningowych
clf.fit(X_train_flat, y_train_binary.flatten())
#Predykcja odpowiedzi modelu na dane testowe
y_pred = clf.predict(X_test_flat)
```

```
c:\Users\jozef\anaconda3\Lib\site-packages\sklearn\svm\_base.py:313: ConvergenceWarning: Solver terminated early (max_iter=10).  Consider pr
e-processing your data with StandardScaler or MinMaxScaler.
  warnings.warn(
```

Dokonujemy obliczeń miar: accuracy, precision i recall.

In [26]:
```
# Obliczenie dokładności klasyfikatora, jak często klasyfikator klasyfikuje poprawnie
print("Miara accuracy:",metrics.accuracy_score(y_test_binary, y_pred))
y_score = clf.decision_function(X_test_flat)
# Obliczenia wartości miary precisson dla modelu: (precission to stosunek wartości prawdziwie pozytywnych wyników do sumy wartości prawdzwie
print("Wartość miary precission dla wytrenowanego modelu:",metrics.precision_score(y_test_binary, y_pred, average="weighted"))
# Obliczenia wartości miary recall dla modelu: (recal to stosunek wartości prawdziwie pozytywnych wyników do sumy wyników prawdziwie pozytyw
print("Wartość miary recall dla wytrenowanego modelu:",metrics.recall_score(y_test_binary, y_pred, average="weighted"))
```

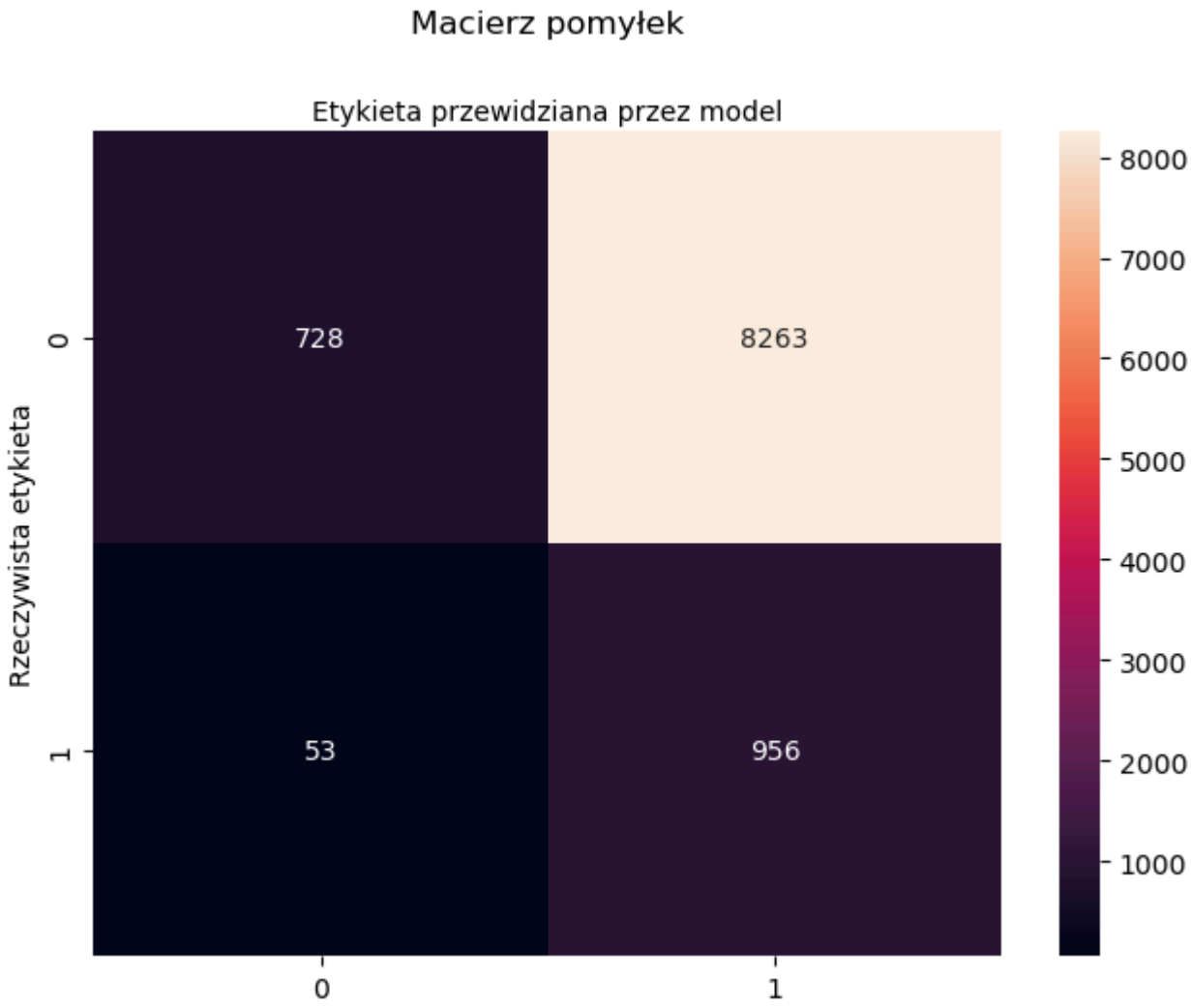
```
Miara accuracy: 0.1684
Wartość miary precission dla wytrenowanego modelu: 0.8485487486387226
Wartość miary recall dla wytrenowanego modelu: 0.1684
```

Po dokonaniu klasyfikacji z użyciem SVM widać, iż metryka accuracy jest na całkiem niezłym poziomie i wynosi ponad 80%. Patrząc tylko na tę metrykę można przypuszczać, iż model sprawuje się całkiem dobrze. Zwracając jednak uwage na pozostałe metryki widać dosyć niepokojące wartości - oscylujące w granicach poniżej 0.16. Oznacza to, iż ze względu na dysproporcję w danych i przewagę ilość danych w jednej klasie klasyfikator zaczyna dokonywać

Dla pewności obliczny jeszcze macierz pomyłek - można będzie bezpośrednio porównać Regresję Liniową z SVM.

In [27]:
```
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = metrics.confusion_matrix(y_test_binary, y_pred)
cnf_matrix
class_names=[0,1] # nazwy klas
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[27]:
```
Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')
```




Dodajmy jeszcze klasyfikator oparty o algortym lasów losowych

In [28]:
```
from sklearn.ensemble import RandomForestClassifier

from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Dokonamy od razu incicjalizacji i predykcji z użcyiem tego algorytmu i otrzymamy miarę accuracy.

In [29]:
```
rf = RandomForestClassifier(n_estimators = 100, random_state = 42)
rf.fit(X_train_flat, y_train_binary.flatten());
predictions_forest = rf.predict(X_test_flat)
```

```
print('Wynik dla miary accuracy:')
accuracy_score(y_test_binary, predictions_forest)
```

```
Wynik dla miary accuracy:
```

Out[29]:
```
0.9844
```

Miara accuracy wygląda bardzo dobrze, szczególnie porównując z pozostałymi klasyfikatorami. A jak wyglądają pozostałe miary?

In [30]:
```
classification_report(y_test_binary, predictions_forest, output_dict= True, target_names=[0,1])
```

Out[30]:
```
{0: {'precision': 0.9844281171181051,
  'recall': 0.9984428873317762,
  'f1-score': 0.9913859745996687,
  'support': 8991.0},
 1: {'precision': 0.9841089670828603,
  'recall': 0.8592666005946482,
  'f1-score': 0.9174603174603174,
  'support': 1009.0},
 'accuracy': 0.9844,
 'macro avg': {'precision': 0.9842685421004826,
  'recall': 0.9288547439632122,
  'f1-score': 0.954423146029993,
  'support': 10000.0},
 'weighted avg': {'precision': 0.9843959148795489,
  'recall': 0.9844,
  'f1-score': 0.9839268757943083,
  'support': 10000.0}}
```

# Zadania do wykonania

**Zadanie 1**

Przeprowadź eksperyment, porównaj klasyfikację z wykorzystaniem regresji logistycznej oraz z SVM i algorytmem lasów losowych. Zmień nastawy klasyfikatora SVM i przetestuj wszystkie funkcje jądra. Zrób podobnie dla lasów losowych wykorzystując wiedzę zdobytą w przykładzie 2. Które nastawy sprawują się najlepiej, a które mają problemy?

**Zadanie 2**

Zmień przyporządkowanie do klas i sprawdź jak wpłynie to na wyniki klasyfikacji z wykorzystaniem SVM i lasów losowych. Zwróć szczególną uwagę na to w jaki sposób zmieniają się metryki po ponownym treningu. Jak wyjaśnisz różnicę między metrykami preccison i recall dla SVM?

In [31]:
```
import time

print("REGRESJA LOGISTYCZNA\n")
```

```python
print("SVM - różne jądra")
kernels = ['linear', 'poly', 'rbf', 'sigmoid']

for k in kernels:
    start_time = time.time()
    clf_test = svm.SVC(kernel=k, max_iter=200, random_state=42)
    clf_test.fit(X_train_flat, y_train_binary.flatten())
    y_pred_svm = clf_test.predict(X_test_flat)
    acc = accuracy_score(y_test_binary, y_pred_svm)
    t = time.time() - start_time
    print(f"SVM kernel='{k}' -> Accuracy: {acc:.4f} (Czas: {t:.2f}s)")

print("\n--- RANDOM FOREST (różne nastawy) ---")

rf_params = [
    {'n_estimators': 50, 'max_depth': 10},
    {'n_estimators': 100, 'max_depth': 5},
    {'n_estimators': 50, 'min_samples_split': 50},
    {'n_estimators': 100, 'min_samples_leaf': 20}
]

for params in rf_params:
    start_time = time.time()
    rf_test = RandomForestClassifier(**params, random_state=42)
    rf_test.fit(X_train_flat, y_train_binary.flatten())
    y_pred_rf = rf_test.predict(X_test_flat)
    acc = accuracy_score(y_test_binary, y_pred_rf)
    t = time.time() - start_time
    print(f"Random Forest {params} -> Accuracy: {acc:.4f} (Czas: {t:.2f}s)")
```

```
REGRESJA LOGISTYCZNA

SVM - różne jądra
c:\Users\jozef\anaconda3\Lib\site-packages\sklearn\svm\_base.py:313: ConvergenceWarning: Solver terminated early (max_iter=200).  Consider p
re-processing your data with StandardScaler or MinMaxScaler.
  warnings.warn(
SVM kernel='linear' -> Accuracy: 0.7876 (Czas: 9.73s)
c:\Users\jozef\anaconda3\Lib\site-packages\sklearn\svm\_base.py:313: ConvergenceWarning: Solver terminated early (max_iter=200).  Consider p
re-processing your data with StandardScaler or MinMaxScaler.
  warnings.warn(
SVM kernel='poly' -> Accuracy: 0.1504 (Czas: 10.22s)
c:\Users\jozef\anaconda3\Lib\site-packages\sklearn\svm\_base.py:313: ConvergenceWarning: Solver terminated early (max_iter=200).  Consider p
re-processing your data with StandardScaler or MinMaxScaler.
  warnings.warn(
SVM kernel='rbf' -> Accuracy: 0.8858 (Czas: 11.19s)
```

```
c:\Users\jozef\anaconda3\Lib\site-packages\sklearn\svm\_base.py:313: ConvergenceWarning: Solver terminated early (max_iter=200).  Consider p
re-processing your data with StandardScaler or MinMaxScaler.
  warnings.warn(
SVM kernel='sigmoid' -> Accuracy: 0.5790 (Czas: 10.50s)

--- RANDOM FOREST (różne nastawy) ---
Random Forest {'n_estimators': 50, 'max_depth': 10} -> Accuracy: 0.9778 (Czas: 13.15s)
Random Forest {'n_estimators': 100, 'max_depth': 5} -> Accuracy: 0.9215 (Czas: 14.58s)
Random Forest {'n_estimators': 50, 'min_samples_split': 50} -> Accuracy: 0.9805 (Czas: 16.60s)
Random Forest {'n_estimators': 100, 'min_samples_leaf': 20} -> Accuracy: 0.9796 (Czas: 30.16s)
```

Komentarz / wnioski do Zadania 1:
Z porównania trzech modeli wynika, że regresja logistyczna radzi sobie bardzo dobrze (prawie 96.8% dokładności).

Random Forest na domyślnych ustawieniach dla 100 drzew osiąga najwyższy wynik – 98.4%. Przy modyfikacji parametrów (np. ograniczenie głębokości drzewa - `max_depth` do 5), skuteczność nieznacznie spada (ok. 92%), co jest efektem zbytniego uproszczenia modelu. Mimo to, wyniki są bardzo zadowalające.

Model SVM w tym przypadku wypada gorzej w kwestii optymalizacji czasu na tak dużym zbiorze. Ze względu na długi czas treningu, liczba iteracji została ograniczona do `max_iter=200`. W efekcie uczenie nie osiągnęło pełnej zbieżności. Jądro `rbf` uzyskało około 88.5% dokładności, natomiast funkcja wielomianowa ( `poly` ) przy takim limicie iteracji poradziła sobie dużo gorzej (15% dokładności). Pokazuje to, że w tym zadaniu algorytm wektorów nośnych wymaga potężnych nakładów obliczeniowych, a algorytm Random Forest jest w tym wymiarze po prostu bardziej praktyczny i elastyczny.

In [33]:
```
# ZADANIE 2: Zmiana klasyfikacji na inny zbiór binarny

y_train_balanced = np.where(y_train < 5, 0.0, 1.0).reshape(y_train.shape[0], 1).astype('float32')
y_test_balanced = np.where(y_test < 5, 0.0, 1.0).reshape(y_test.shape[0], 1).astype('float32').flatten()

print("SVM (wyważone klasy)")
clf_balanced = svm.SVC(C=10, kernel='linear', probability=False, max_iter=300, random_state=42)
clf_balanced.fit(X_train_flat, y_train_balanced.flatten())
y_pred_svm_balanced = clf_balanced.predict(X_test_flat)

print("Miara accuracy SVM:", metrics.accuracy_score(y_test_balanced, y_pred_svm_balanced))
print("Miara precision SVM:", metrics.precision_score(y_test_balanced, y_pred_svm_balanced, average="weighted"))
print("Miara recall SVM:", metrics.recall_score(y_test_balanced, y_pred_svm_balanced, average="weighted"))

print("\n--- RANDOM FOREST (wyważone klasy) ---")
rf_balanced = RandomForestClassifier(n_estimators=100, random_state=42)
rf_balanced.fit(X_train_flat, y_train_balanced.flatten())
y_pred_rf_balanced = rf_balanced.predict(X_test_flat)

print("Miara accuracy Random Forest:", metrics.accuracy_score(y_test_balanced, y_pred_rf_balanced))
```

```
print("Miara precision Random Forest:", metrics.precision_score(y_test_balanced, y_pred_rf_balanced, average="weighted"))
print("Miara recall Random Forest:", metrics.recall_score(y_test_balanced, y_pred_rf_balanced, average="weighted"))
```

```
SVM (wyważone klasy)
```

```
c:\Users\jozef\anaconda3\Lib\site-packages\sklearn\svm\_base.py:313: ConvergenceWarning: Solver terminated early (max_iter=300).  Consider p
re-processing your data with StandardScaler or MinMaxScaler.
  warnings.warn(
Miara accuracy SVM: 0.6672
Miara precision SVM: 0.6699689509312284
Miara recall SVM: 0.6672

--- RANDOM FOREST (wyważone klasy) ---
Miara accuracy Random Forest: 0.9762
Miara precision Random Forest: 0.9762357839487393
Miara recall Random Forest: 0.9762
```

Komentarz do Zadania 2

W tym ćwiczeniu klasy zostały przydzielone w sposób zbilansowany z 0-8 vs 9 (proporcja wynosiła 90%:10%) na liczby 0-4 vs liczby 5-9 (proporcja ~50%:50%).

Dla klasyfikatora Random Forest dokładność po przeprowadzonej zmianie wciąż była bardzo dobra i wynosiła na testach ~97.6%. Zarówno uśrednione miary Precision, jak i Recall dla obu klas ustabilizowały się w tych zrównoważonych rozkładach i wskazywały po 97.6%.

Klasyfikator SVM: Przy starszym, bardzo niezbalansowanym zadaniu klasyfikacyjnym model SVM, aby najszybciej osiągnąć wysoką mierzoną dokładność, miał tendencję do klasyfikowania niemal wszystkiego z etykietą głównej klasy większościowej. Wtedy model miał wysoką dokładność ogólną, lecz niską precyzję i czułość dla rzadszej klasy mniejszościowej.
Obecnie, mając rozkład prób po równo 50/50, widzimy rzeczywiste, uśrednione możliwości modelu przy ucięciu iteracji. Gdy zbiór jest zbilansowany, te dysproporcje miedzy precision, recall i dokładnością już tak nie skaczą i odzwierciedlają prawdziwy (niższy, ok 66%) poziom wytrenowania optymalizatora SVM przy ograniczonym do 300 liczbie cyklów. SVM w środowisku podobnym wielkościowo do MNIST znacznie przegrywa efektywnością z lasami ze względu na czas szkolenia.

Fundusze Europejskie Polska Cyfrowa
Rzeczpospolita Polska
Unia Europejska Europejski Fundusz Rozwoju Regionalnego